Fundamentals of Programming Languages

Assignment 2

Gödel's System T

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

You must deliver two solutions: one in SASyLF🏹 for section 1, the other in Haskell🏹 for section 2.

1 Gödel's system T

Gödel's system T extends the simply typed lambda calculus with natural numbers and a *recursor*. The recursor defines a function on all natural numbers from its result on zero and from how to obtain its result on $n + 1$ from n and from its result on n. This is the computational counterpart of induction on natural numbers.

The syntax of the simply typed lambda calculus (TAPL, Figure 9-1) is extended with natural numbers, which also provide the base type, and with a recursor.

t	$::=$	*terms:*	v	$::=$	*values:*
	x	*variable*		$\lambda x\colon T.t$	*abstraction value*
	$\lambda x\colon T.t$	*abstraction*		nv	*numeric value*
	$t\,t$	*application*	nv	$::=$	*numeric values:*
	$\mathtt{0}$	*zero*		$\mathtt{0}$	*zero value*
	$\mathtt{succ}\ t$	*successor*		$\mathtt{succ}\ nv$	*successor value*
	$\mathtt{rec}(t; x.y.t)(t)$	*recursor*	T	$::=$	*types:*
				$\mathtt{Nat}$	*type of natural numbers*
				$T \to T$	*type of functions*

The expression $\mathtt{rec}(t_0; x.y.t_1)(t)$ is called a recursor. The bound variable x represents the predecessor and the bound variable y the result of the recursive call on x.

The evaluation of the recursor is governed by three rules.

$$\frac{t_2 \;\to\; t_2'}{\texttt{rec}(t_0; x.y.t_1)(t_2) \;\to\; \texttt{rec}(t_0; x.y.t_1)(t_2')}\ \text{E-RecA}$$

$$\frac{}{\texttt{rec}(t_0; x.y.t_1)(\texttt{0}) \;\to\; t_0}\ \text{E-RecZ}$$

$$\frac{}{\texttt{rec}(t_0; x.y.t_1)(\texttt{succ}\, nv) \;\to\; [x \mapsto nv][y \mapsto \texttt{rec}(t_0; x.y.t_1)(nv)]t_1}\ \text{E-RecS}$$

Rules E-RecZ and E-RecS specify the behavior of the recursor on `0` and on `succ` nv, where nv stands for a numeric value. In the former case, the recursor selects term t_0—the base of the recursion; in the latter case it selects term t_1—the step—and binds x to the predecessor (that is nv) and y to the result of the recursion on nv. Rule E-RecA evaluates the argument. This rule is applied as many times as needed, until the argument becomes a value. Thereafter rule E-RecZ or E-RecS apply, depending on whether the argument (a numeric value) is zero or not.

The evaluation rules for the typed lambda calculus with arithmetic can be found in TAPL, Figures 3-2 (excluding the rules for `pred` and `iszero`, which are not part of our syntax) and 9-1.

For example, a function that doubles its input can be written in equation format as follows.

$$\begin{aligned}\texttt{double}\,\texttt{0} &= \texttt{0}\\ \texttt{double}\,(\texttt{succ}\,n) &= \texttt{succ}\,(\texttt{succ}\,(\texttt{double}\,n))\end{aligned}$$

Using the recursor we write instead (notice how y represents the result of evaluating `double` n):

$$\texttt{double} = \lambda n\colon \texttt{Nat}.\texttt{rec}(\texttt{0}; x.y.\texttt{succ}\,(\texttt{succ}\,y))(n)$$

In this case we have not used the predecessor (the bound variable x); the simplified form of recursor, that does not use the predecessor, is usually called an iterator.

Notes on SASyLF.

- *Binding.* The recursor binds two variables in its step. Write `rec t0 { x . x' . t1[x][x'] } t2` for $\texttt{rec}(t_0; x.x'.t_1)(t_2)$, so that $[x \mapsto t][x' \mapsto s]t_1$ is written `t1[t][s]`.
- *Rule E-RecS.* SASyLF accepts only metavariables as the arguments of a substitution, as in `t1[t][t2]`: it rejects `t1[t][rec ...]`. Rule E-RecS can instead name the recursive call `t2`, with a premise `t2 = rec t0 { x . x' . t1[x][x'] } t` stating what `t2` is, where `t = t'` is a judgment for term equality with a single rule, reflexivity.
- *Parentheses.* SASyLF does not know that application associates to the left, nor how far the body of an abstraction extends: write `(t1 t2) t3` for $t_1\, t_2\, t_3$, and put parentheses around the body of an abstraction when it is an application or a recursor.

A. Example of reduction. Writing 2 in place of $\texttt{succ}\,(\texttt{succ}\,\texttt{0})$ and similarly for 4, show that

$$\texttt{double}\;2 \;\to^*\; 4.$$

Note: $t \;\to^*\; t'$ is the reflexive and transitive closure of relation $t \;\to\; t'$, meaning that t reduces to t' in zero or more steps.

B. A definition for `sum`. Taking advantage of the recursor, write the `sum` operator in the form of a function $\lambda n\colon \texttt{Nat}.\lambda m\colon \texttt{Nat}.t$. Then show that $\texttt{sum}\;2\;4 \;\to^*\; 6$.

C. Typing the recursor. Propose a typing rule T-Rec for the recursor.

D. Typing derivation. Show that term `double` above is typable under the empty context, that is $\vdash \texttt{double} : T$, for an appropriate T that you are to discover. Likewise for `sum`.

E. Predecessor. Write a term `pred` for the predecessor function, so that $\texttt{pred}\;\texttt{0}$ reduces to `0` and $\texttt{pred}\;(\texttt{succ}\;nv)$ reduces to nv, and show that it is typable. Leave the syntax of terms as is: `pred` is a term written with the recursor, not a new term constructor. Then prove that $\texttt{pred}\;(\texttt{succ}\;nv) \;\to^*\; nv$ for *every* numeric value nv, not only for an example. How many steps does $\texttt{pred}\;n$ take, compared with $\texttt{double}\;n$? Why?

F. Ackermann's function. Ackermann's function is given by the following equations.

$$\begin{aligned}
\texttt{ack}\;\texttt{0}\;n &= \texttt{succ}\;n \\
\texttt{ack}\;(\texttt{succ}\;m)\;\texttt{0} &= \texttt{ack}\;m\;1 \\
\texttt{ack}\;(\texttt{succ}\;m)\;(\texttt{succ}\;n) &= \texttt{ack}\;m\;(\texttt{ack}\;(\texttt{succ}\;m)\;n)
\end{aligned}$$

Ackermann's function is not *primitive recursive*, that is, it is not definable when recursors return numbers only: it grows faster than any such function. Yet it is definable in Gödel's system T. Write a term `ack` for Ackermann's function, and show that $\vdash \texttt{ack} : \texttt{Nat} \to \texttt{Nat} \to \texttt{Nat}$. Suggestion: write $\texttt{ack}\;m$, a function of type $\texttt{Nat} \to \texttt{Nat}$, by recursion on m; for the step, use an inner recursion on n. What is the type of the outer recursor? Why is this not possible with first-order primitive recursion, where recursors only return numbers?

G. Progress. Show the following result:

> [Progress]: Suppose t is a well-typed, closed term (that is, $\vdash t : T$ for some T). Then either t is a value or else there is some t' such that $t \;\to\; t'$.

Address all cases, not only those for the recursor. You may need canonical forms lemmas for type `Nat` (compare with Lemma 8.3.1) and for function types (Lemma 9.3.4). Prove the lemma for `Nat`; for function types, a case analysis on the value suffices.

H. Determinacy. Prove that evaluation is deterministic: if $t \;\to\; t'$ and $t \;\to\; t''$, then $t' = t''$. You will need a lemma stating that values do not evaluate (compare with Theorems 3.5.4 and 3.5.7).

I. Preservation. Show the following result:

[Preservation]: If $\Gamma \vdash t : T$ and $t \to t'$ then $\Gamma \vdash t' : T$.

Lemma 9.3.8 (Preservation of types under substitution) is built in SASyLF; it works for the two variables bound by the recursor as well.

2 Implementation

Adapt and extend the implementation of the simply typed lambda calculus given in class to Gödel's system T (section 1).

A. The datatypes for types and terms. Adapt and extend the datatypes `Type` and `Term`. Write `Rec t0 x y t1 t2` for the recursor $\texttt{rec}(t_0; x.y.t_1)(t_2)$. Define terms `one`, `two`, `three` and `four` for the numerals 1 to 4; `double` (as in page 2) and `doubleTwo` for `double` 2; `sum` (your answer to item 1.B) and `sumTwoFour` for `sum` 2 4.

B. Typing. Adapt and extend the typing function.

```
typeof :: Context → Term → Type
```

The case for `Rec` implements your rule T-Rec (item 1.C). Make sure that `double`, `doubleTwo`, `sum` and `sumTwoFour` have the expected types, and compare with your answer to item 1.D.

C. Single step evaluator. Adapt and extend the single step evaluator.

```
step :: Term → Term
```

You may have to extend other functions such as `freeVars` and `subs`. Take

$$\texttt{FV}(\texttt{rec}(t_0; x.y.t_1)(t_2)) = \texttt{FV}(t_0) \cup (\texttt{FV}(t_1) \setminus \{x, y\}) \cup \texttt{FV}(t_2)$$

Using function `eval`, show that `doubleTwo` and `sumTwoFour` evaluate to the expected values, and compare with your answers to items 1.A and 1.B.

D. More functions. Write terms `pred` (your answer to item 1.E), `times` for the product (using `sum`) and `fact` for the factorial (using `times`). Check their types with `typeof`.

E. Ackermann's function. Write in Haskell your term `ack` of item 1.F, using the constructor `Rec`, and check that `typeof` gives it type `Nat → Nat → Nat`. Then define terms `ackOneTwo` for `ack` 1 2 and `ackTwoTwo` for `ack` 2 2, and use `eval` to show that they evaluate to 4 and 7, respectively.

Due date: November 7, 2026, 23:59.